



the rule of three

Level : 2A

Team C++

AY: 2020/2021



Outline of the chapter



Reminder: Collection of polymorphic objects



Redefining the destructor



Redefinition of the copy constructor



Redefinition of the assignment operator



To remember

Reminder: Collection of polymorphic objects

main.cpp

```
int main()
{
    Account C1(123, 1000);
    C1.display();
    cout<<"#####"<<endl;

    CurrentAccount CCl(234,2000);
    CCl.display();
    cout<<"#####"<<endl;

    SavingAccount CE;
    CE.display();
    cout<<"#####"<<endl;

    Bank B;

    cout<<"##### polymorphism #####"<<endl;
    B.add(CCl);
    B.add(C1);
    B.add(CE);

    cout<<"##### Banque B #####"<<endl;
    B.display();
}
```

Main and object B must not manipulate the same variables!

→ The variables of the main added in the bank B

Reminder: Collection of polymorphic objects

main.cpp

```
int main()
{
    Account C1(123, 1000);
    C1.display();
    cout<<"#####"<<endl;

    CurrentAccount CC1(234, 2000);
    CC1.display();
    cout<<"#####"<<endl;

    SavingAccount CE;
    CE.display();
    cout<<"#####"<<endl;

    Bank B;

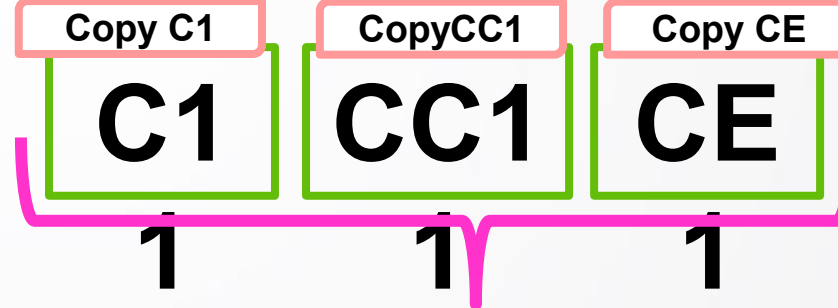
    cout<<"##### polymorphism #####"<<endl;
    B.add(CC1);
    B.add(C1);
    B.add(CE);

    cout<<"##### Banque B #####"<<endl;
    B.display();
}
```

```
bool Bank::add (SavingAccount const & C)
{
    if (searching(C.GetRIB()) != tab.end())
    {
        return false;
    }

    Account *p=new SavingAccount(C);
    tab.push_back(p);

    return true;
}
```



Add **copies** of C1, CC1 and CE (variables of the main) in object B (C11, CC11, CE1)

Redefinition of the destructor

main.cpp

```
int main()
{
    Account C1(123, 1000);
    C1.display();
    cout<<"#####"<<endl;

    CurrentAccount CC1(234,2000);
    CC1.display();
    cout<<"#####"<<endl;

    SavingAccount CE;
    CE.display();
    cout<<"#####"<<endl;

    Bank B;

    cout<<"##### polymorphism #####"
    B.add(CC1);
    B.add(C1);
    B.add(CE);

    cout<<"##### Banque B #####"
    B.display();
}
```

Static
Allocation

Call to the **destructor** of
the Bank class

Bank B; // Instantiation of the object B

B

Name : « »

Adress : « »

tab :

@C1 @CC1 @CE

C1

CC1

CE

These **are copies** of C1, CC1 and CE

Redefinition of the destructor

bank.h

```
~Bank();
```

bank.cpp

```
Bank::~~Bank()  
{  
    cout<<"I am the destroyer of the Bank class"<<endl;  
}
```

main.cpp

I am the destroyer of the Bank class

Redefinition of the destructor

Banque B:

Call to the **destructor** of the Bank class

B.add(C1);

B.add(CC1);

B.add(CE);

B.Display();

return 0;

- Call of the destructor of the class **String**
- Call of the destructor of the class **String**
- Call of the destructor of the class **Vector**

B

This memory space will **not be released!**

Adress : «

tab :

@C1

1

C1

1

@CE

1

C1

1

CC1

1

CE

1



Free this space in the **destructor** of the Bank class



Redefinition of the destructor

bank.h

```
~Bank();
```

bank.cpp

```
Bank::~~Bank()  
{  
    cout<<"I am the destroyer of the Bank class"<<endl;  
}
```



```
Bank::~~Bank()  
{  
    cout<<"I am the destroyer of the Bank class"<<endl;  
  
    vector<Account*>::iterator i ;  
  
    for(i=tab.begin(); i!=tab.end(); i++)  
    {  
        delete (*i);  
    }  
}
```

Redefinition of the destructor

bank.cpp

```
Bank::~Bank()  
{  
  
    cout<<"I am the destroyer of the Bank class"<<endl;  
  
    vector<Account*>::iterator i ;  
    for(i=tab.begin(); i!=tab.end(); i++)  
    {  
        delete (*i);  
    }  
}
```

main.cpp

```
I am the destructor of the class Account  
I am the destructor of the class Current account  
I am the destructor of the class Account  
I am the destructor of the class Current account  
I am the destructor of the class Account
```

Redefinition of the destructor

Banque B:

B.add(C1);

B.add(CC1);

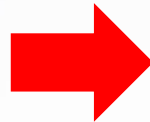
B.add(CE);

B.Display();

return 0;

}

Call of the **destructor** of the class Bank



- **delete (*i);**
- Call of the destructor of the class **String**
- Call of the destructor of the class **String**
- Call of the destructor of the class **Vector**

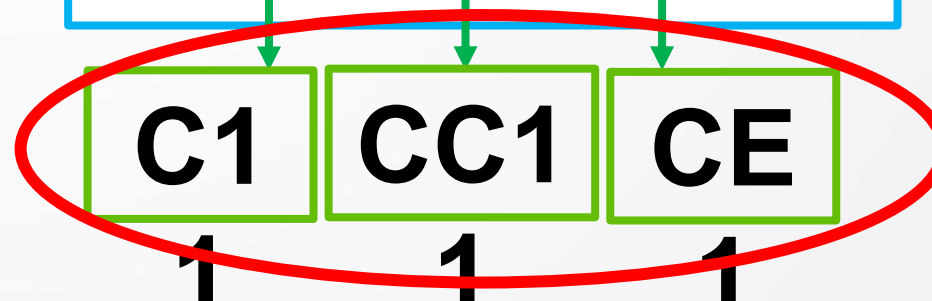
B

Name : « »

Adress : « »

tab :

@C1 @CC1 @CE
1 1 1



Redefinition of the copy constructor

main.cpp

```
cout<<"##### Bank B #####"<<endl;
B.Display();

cout<<"##### Constructor by copy #####"<<endl;

Banque A(B);

A.Display();
```

Call **to the copy constructor** of the Bank class

```
##### Bank B #####
RIB : 234
Pay : 1.29051e-306
RIB : 123
Pay : 1.29035e-306
RIB : 0
Pay : 5
rate : 0
##### Constructor of copy #####
RIB : 234
Pay : 1.29051e-306
RIB : 123
Pay : 1.29035e-306
RIB : 0
Pay : 5
rate : 0
```

Original

COPY

Redefinition of the copy constructor

main.cpp

```
Bank A(B);  
B.modify(123);  
cout<<"##### Bank B #####"<<endl;  
B.Display();  
cout<<"##### Bank A copy of Bank B #####"<<endl;  
A.Display();
```

Modification of the content of Bank B

bank.cpp

```
bool Banque::modifier (long RIB) const  
{  
    vector <Compte*>::const_iterator i ;  
    for(i=tab.begin(); i!=tab.end(); i++)  
    {  
        if((**i).GetRIB()== RIB)  
        {  
            (**i).Setsolde(55555);  
            return true;  
        }  
    }  
    return false;  
}
```

Redefinition of the copy constructor

main.cpp

```
Bank A(B);
```

```
B.modify();
```

Modification of the content of Bank B

```
cout<<"##### Bank B #####"<<endl;
```

```
B.Display();
```

```
cout<<"##### Bank A copy of Bank B #####"<<endl;
```

```
A.Display();
```

```
##### Banque B #####
```

```
RIB : 123
```

```
Solde : 55555
```

```
RIB : 234
```

```
Solde : 2000
```

```
Seuil : 200
```

```
RIB : 0
```

```
Solde : 5
```

```
taux : 0
```

```
##### La banque A copie de B #####
```

```
RIB : 123
```

```
Solde : 55555
```

```
RIB : 234
```

```
Solde : 2000
```

```
Seuil : 200
```

```
RIB : 0
```

```
Solde : 5
```

```
taux : 0
```

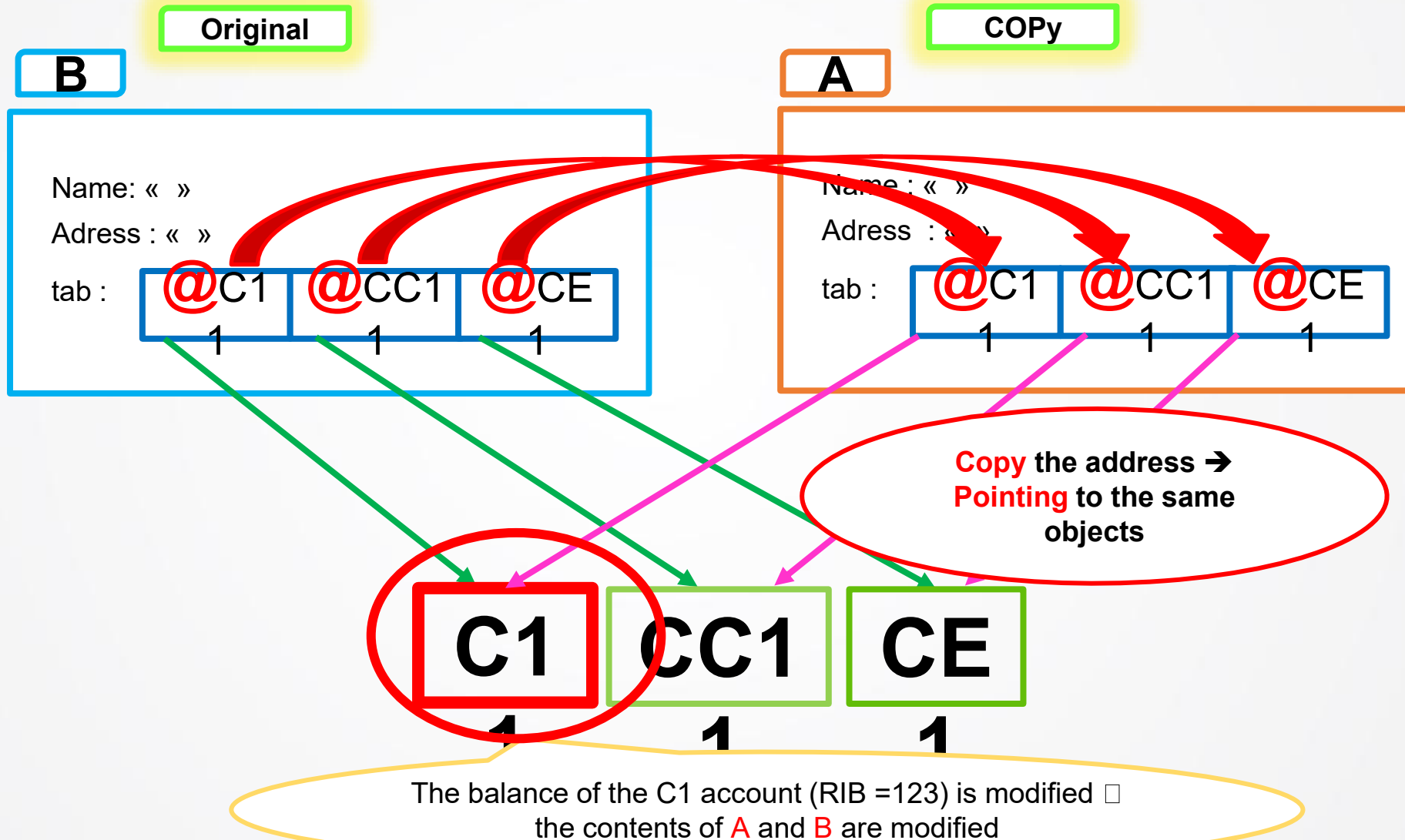
B.modify();

Original Modified

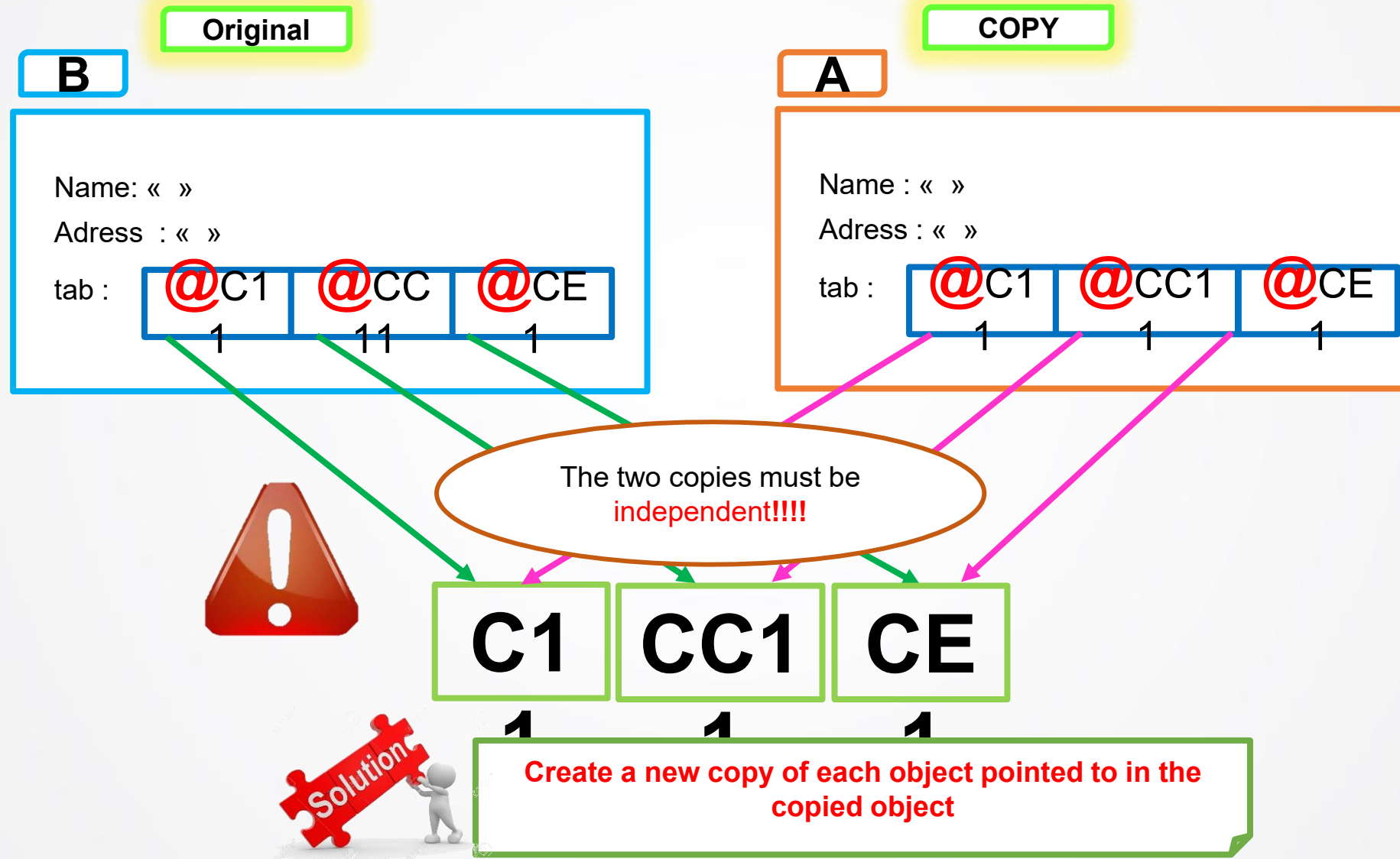
Copy Modified !!!

?

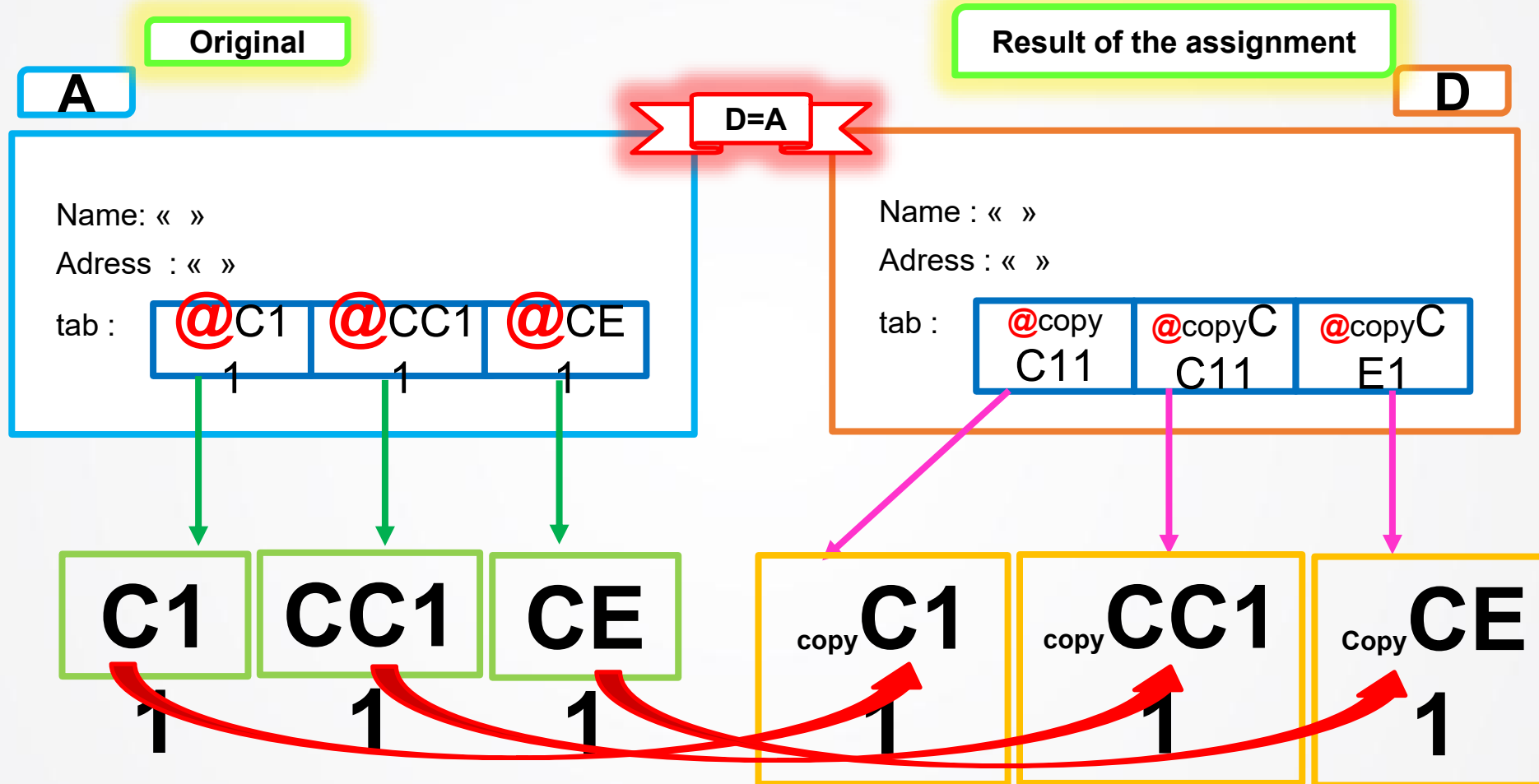
Redefinition of the copy constructor



Redefinition of the copy constructor



Redefinition of the assignment operator



Redefinition of the assignment operator

bank.h

```
Bank&operator=(const Bank&B)
```

bank.cpp

```
Bank::Bank(const Bank &B)
{
    Account *p;
    for(vector<Account *>::const_iterator i=B.tab.begin(); i != B.tab.end(); i++)
    {
        if(typeid(**i)==typeid(Account))
        {
            p=new Account (**i);
        }
        else if(typeid(**i)==typeid(savingsaccount))
        {
            p=new savingsaccount(static_cast<const savingsaccount&> (**i));
        }
        else
        {
            p=new currentaccount(static_cast<const currentaccount&>(**i));
        }
        tab.push_back(p);
    }
    return *this;
```

Creation of new
copies

Constructor by copy

Expl : A = B =C ;

Redefinition of the assignment operator

main.cpp

```
A.modifier(234);
```

Modification of the content of Bank A

```
cout<<"##### Banque A #####"<<endl;
A.afficher();
cout<<"##### Banque D #####"<<endl;
D.afficher();
```

```
##### Bank A #####
RIB : 234
Pay : 55555
RIB : 123
Pay : 55555
RIB : 0
Pay : 5
rate : 0
##### Banque D #####
RIB : 234
Pay : 1.29051e-306
RIB : 123
Pay : 55555
RIB : 0
Pay : 5
rate : 0
```

Original

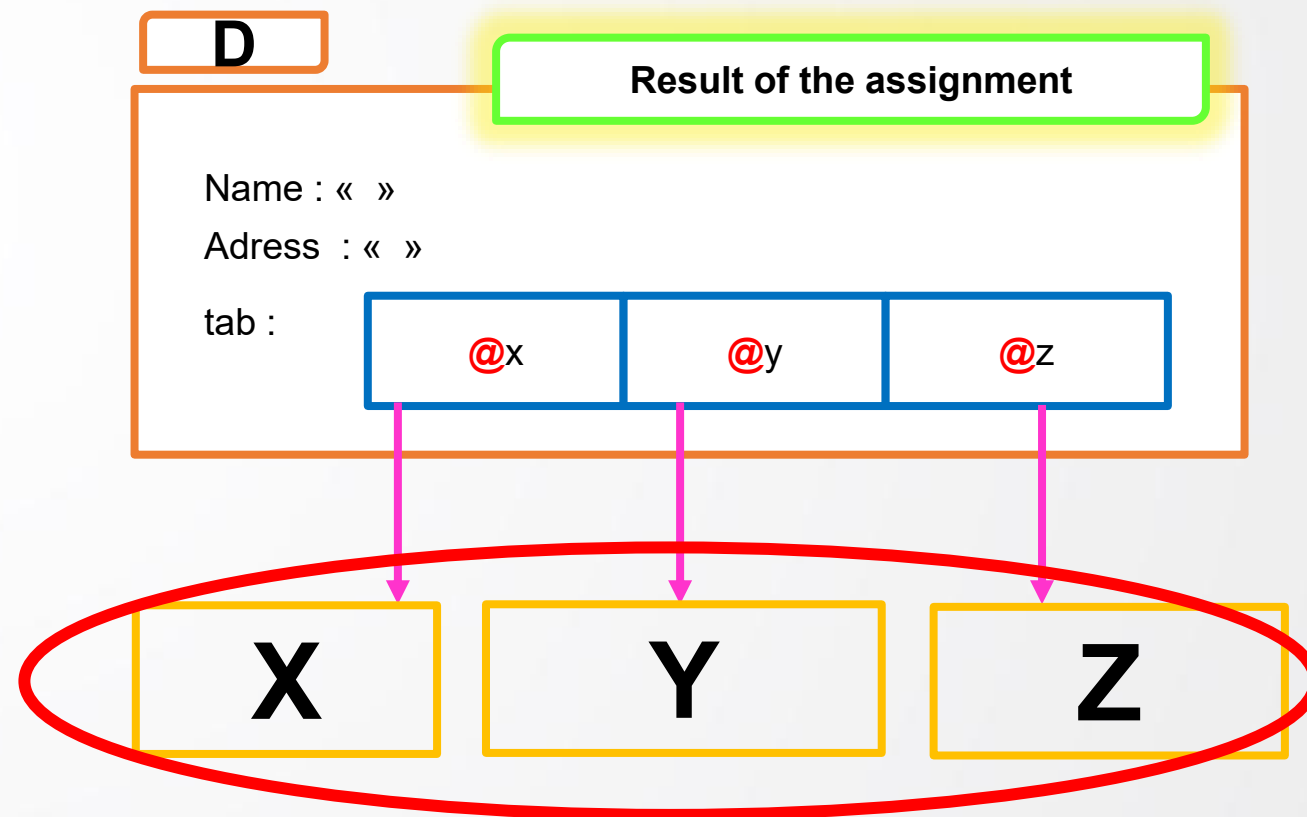
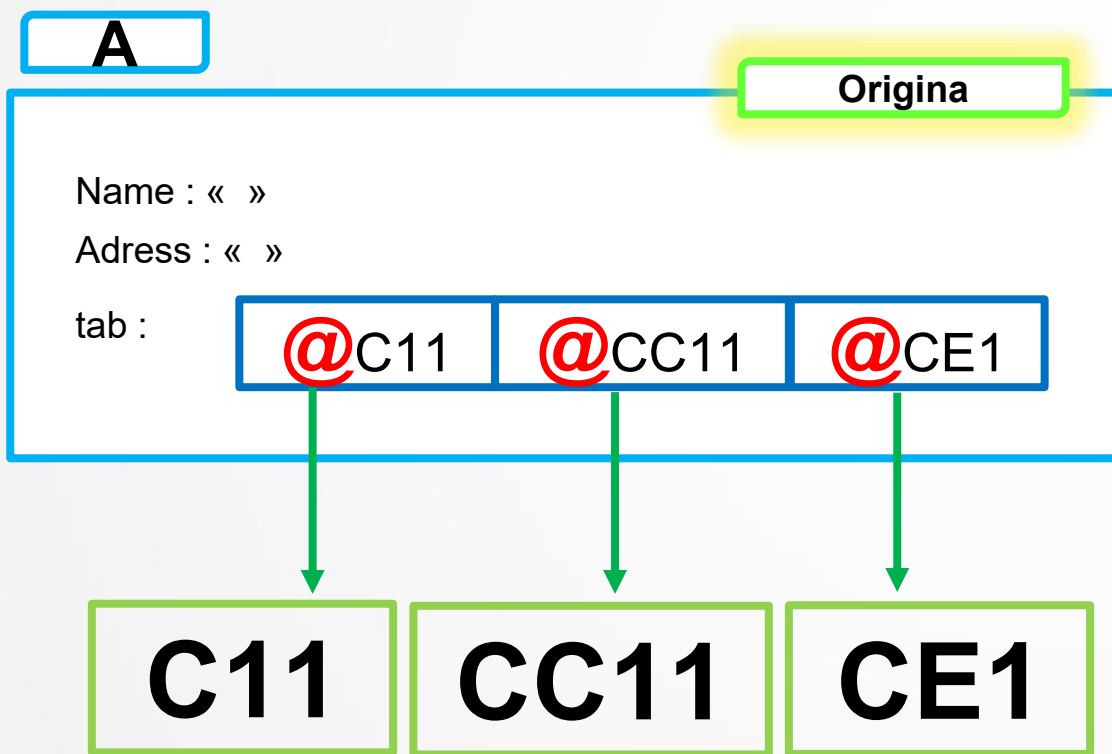
A.modifier();

Result of the assignment

Bank D contents **NOT** modified

► Redefinition of the assignment operator

If D already references other objects ... !!





Redefinition of the assignment operator



main.cpp

```
cout<<"##### Assignment A in D #####"<<endl;
```

```
Bank D;
```

```
Account X;
```

```
currentaccount Y (456,33);
```

```
currentaccount Z (22222,2000);
```

```
D.add(X);
```

```
D.add(Y);
```

```
D.add(Z);
```

```
cout<<"##### bank D #####"<<endl;
```

```
D.display();
```

```
D=A; //D.operator=(A);
```

```
A.change(234);
```

```
cout<<"##### Bank A #####"<<endl;
```

```
A.display();
```

```
cout<<"##### Banque D #####"<<endl;
```

```
D.display();
```

D contains copies of objects
X, Y et Z

assign **A** in **D**

Redefinition of the assignment operator

```
##### bank D #####  
RIB : 0  
Pay : 0  
RIB : 456  
Pay : 5.16106e-305  
RIB : 22222  
Pay : 6.28154e-317
```

D contains copies of objects
X, Y et Z

```
Bank D;  
  
Account X;  
savingsaccount y;  
y.enter();  
currentaccount Z (22222,2000,200);  
  
D.add(X);  
D.add(Y);  
D.add(Z);  
  
D.display();
```

```
##### Bank A #####  
RIB : 234  
Pay : 55555  
RIB : 123  
Pay : 55555  
RIB : 0  
Pay : 5  
rate : 0
```

Original

Result of the assignment

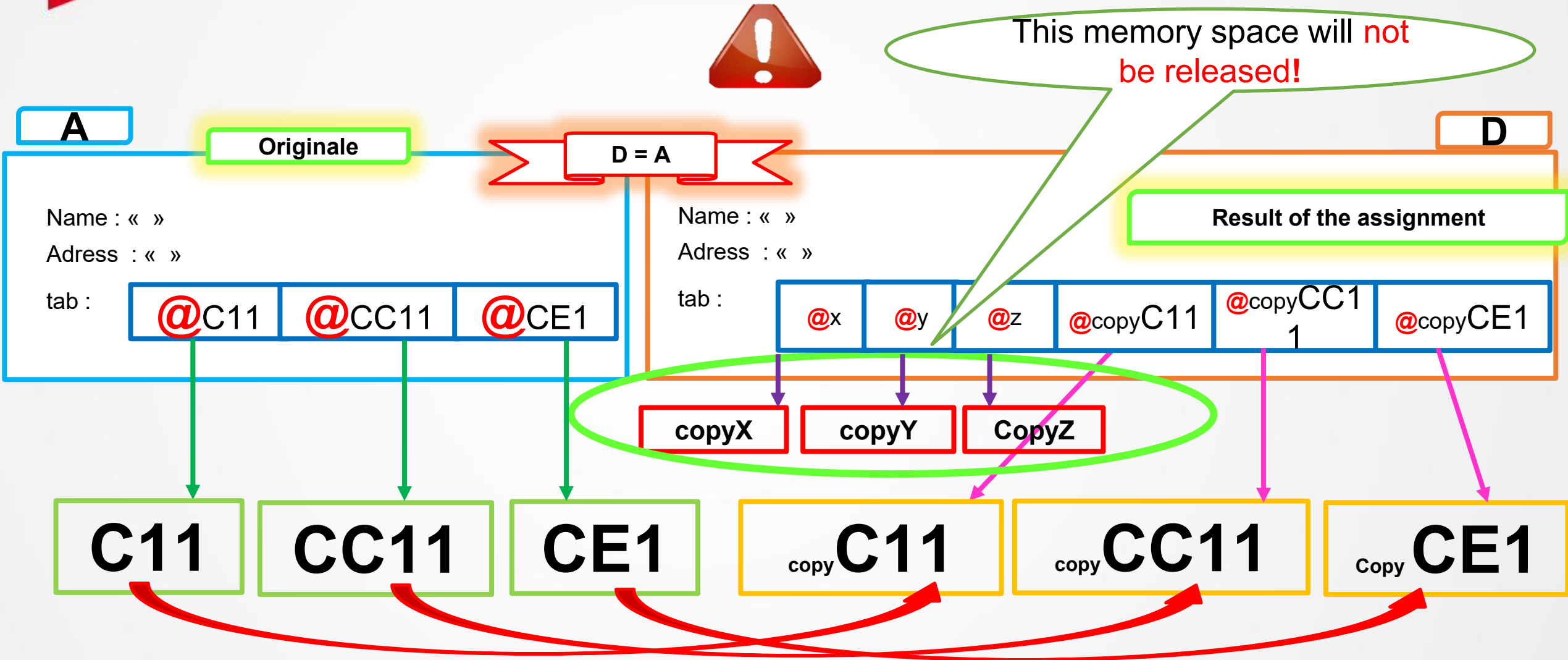
```
D=A; //D.operator=(A);  
  
A.change(234);  
A.display();  
D.display();
```

```
##### Banque D #####  
RIB : 234  
Pay : 1.29051e-306  
RIB : 123  
Pay : 55555  
RIB : 0  
Pay : 5  
rate : 0
```

Result of the assignment **A** in **D**
D contains copies of
X, Y , Z C11, CC11, CE1 !!!

?

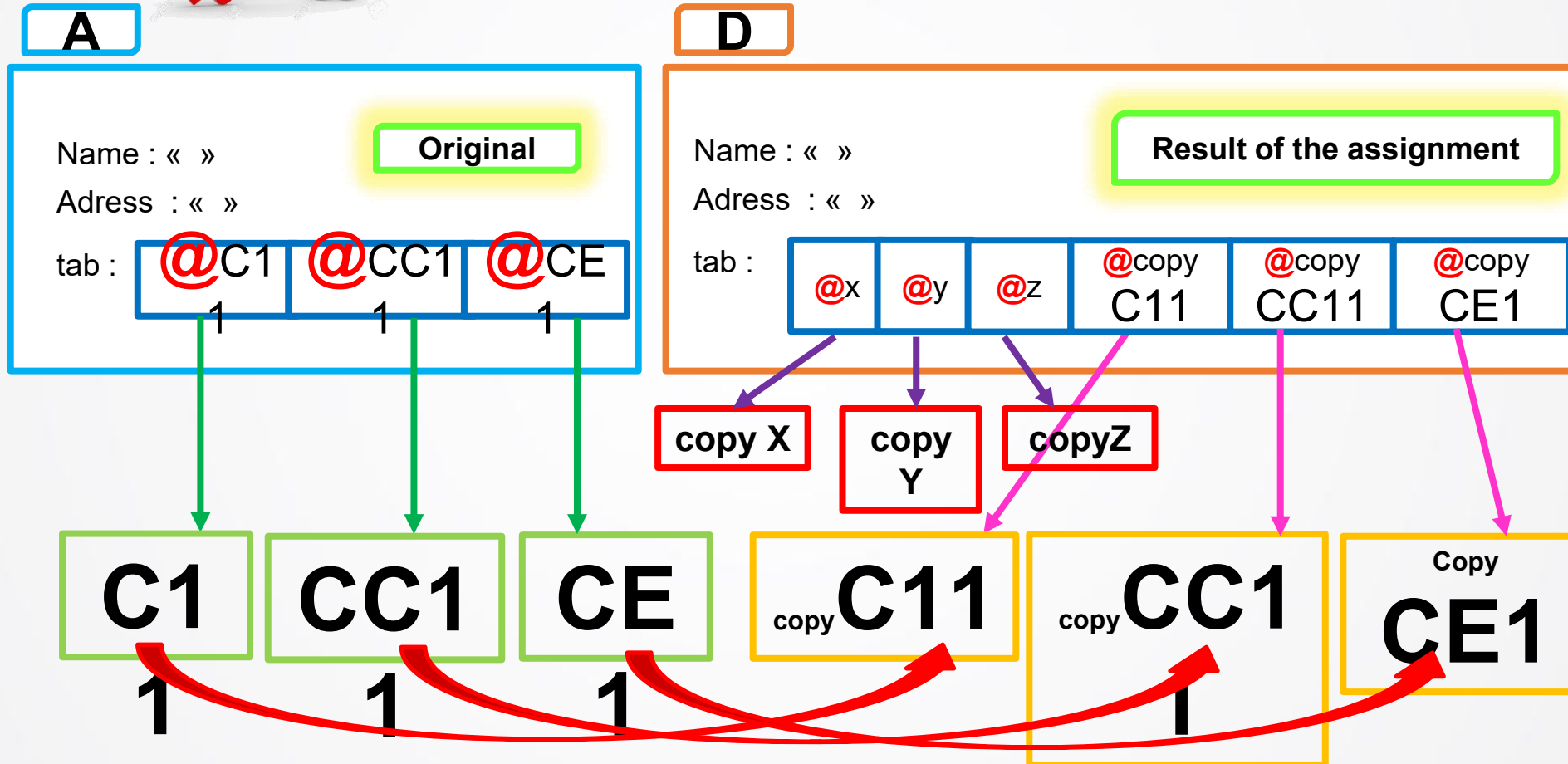
Redefinition of the assignment operator



Redefinition of the assignment operator



Delete the old content of D before making the assignment



Redefinition of the assignment operator

bank.cpp

```
Bank& Bank::operator=(const Bank&B)
{
    if(&B!=this)
    {
        vector<Account*>::iterator i ;
        for(i=tab.begin(); i!=tab.end(); i++)
        {
            delete (*i);
        }
        tab.clear();

        Account*p;
        for(vector<Account*>::const_iterator i=B.tab.begin(); i != B.tab.end(); i++)
        {
            if(typeid(**i)==typeid(Account))
            {
                p=new Account(**i);
            }
            else if(typeid(**i)==typeid(currentaccount))
            {
                p=new currentaccount(static_cast<const currentaccount&>(**i));
            }
            else
            {
                p=new savingsaccount(static_cast<const savingsaccount&>(**i));
            }
            tab.push_back(p);
        }
        return *this;
    }
}
```

Destructor

Redefinition of the assignment operator

bank.cpp

```
Bank& Bank::operator=(const Bank&B)
```

```
{  
    if(&B!=this)
```

```
{
```

```
    vector<Account*>::iterator i ;  
    for(i=tab.begin(); i!=tab.end(); i++)  
    {  
        delete (*i);  
    }
```

```
    tab.clear();
```

```
    Account*p;
```

```
    for(vector<Account*>::const_iterator i=B.tab.begin(); i != B.tab.end(); i++)  
    {  
        if(typeid(**i)==typeid(Account))  
        {  
            p=new Account(**i);  
        }  
        else if(typeid(**i)==typeid(currentaccount))  
        {  
            p=new currentaccount(static_cast<const currentaccount&>(**i));  
        }  
        else  
            p=new savingsaccount(static_cast<const savingsaccount&>(**i));  
        tab.push_back(p);  
    }
```

```
    return *this;  
}
```

Avoid assigning an object to itself

Expl : A = A !!!

Destructor

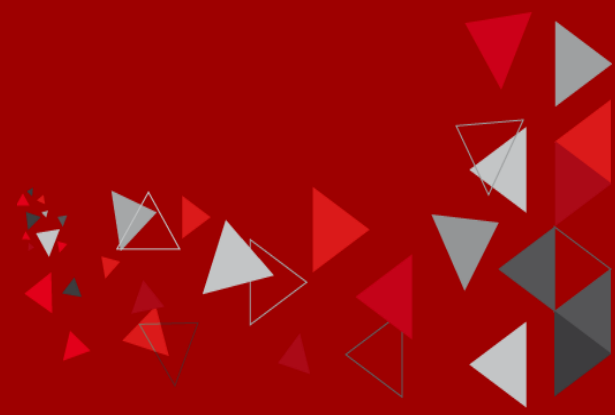
Constructor of copy

Expl : A = B =C ;



► To remember

- « Defining **a copy constructor** for a class usually means that **the copy constructor, destructor and assignment operator** provided by default by the compiler **are not suitable for this class**. Therefore, all three methods will have to be systematically redefined as soon as one of them is. » *Course of C/C++, Christian Casteyde*
- « This rule, called **the rule of three**, will allow you to avoid bugs easily. » *Course of C/C++, Christian Casteyde*
- « This will be the case when some of the data of the objects have been **dynamically allocated**. A brutal copy of the fields of one object into another would only copy the pointers, **not the pointed data**. Thus, changing the data for one object would change the data for the other object, which would probably not be the desired effect. » *Course of C/C++, Christian Casteyde*



THANK YOU

Any Questions
?